



# Java HashMap

[< Previous](#)[Next >](#)

## Java HashMap

A **HashMap** stores items in **key/value pairs**, where each key maps to a specific value.

It is part of the **java.util** package and implements the **Map** interface.

Instead of accessing elements by an index (like with **ArrayList**), you use a **key** to retrieve its associated **value**.

A **HashMap** can store many different combinations, such as:

- **String** keys and **Integer** values
- **String** keys and **String** values

## Create a HashMap

Create a **HashMap** object called **capitalCities** that will store **String** keys and **String** values:

## Example

[Get your own Java Server](#)



Now you can use methods like `put()` to add key/value pairs, `get()` to retrieve a value by key, and `remove()` to delete an entry - all by using keys instead of index numbers.

## Add Items

To add items to a `HashMap`, use the `put()` method:

### Example

```
// Import the HashMap class
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        // Create a HashMap object called capitalCities
        HashMap<String, String> capitalCities = new HashMap<String, String>();

        // Add keys and values (Country, City)
        capitalCities.put("England", "London");
        capitalCities.put("India", "New Dehli");
        capitalCities.put("Austria", "Wien");
        capitalCities.put("Norway", "Oslo");
        capitalCities.put("Norway", "Oslo"); // Duplicate
        capitalCities.put("USA", "Washington DC");

        System.out.println(capitalCities);
    }
}
```



**Note:** In the example above, if the same key (like "Norway") is added more than once, the *latest* value will overwrite the previous one, because keys in a **HashMap** must be unique.

## Access an Item

To access a value in the **HashMap**, use the **get()** method and refer to its key:

### Example

```
capitalCities.get("England");
```

[Try it Yourself »](#)

## Remove an Item

To remove an item, use the **remove()** method and refer to the key:

### Example

```
capitalCities.remove("England");
```

[Try it Yourself »](#)

To remove all items, use the **clear()** method:



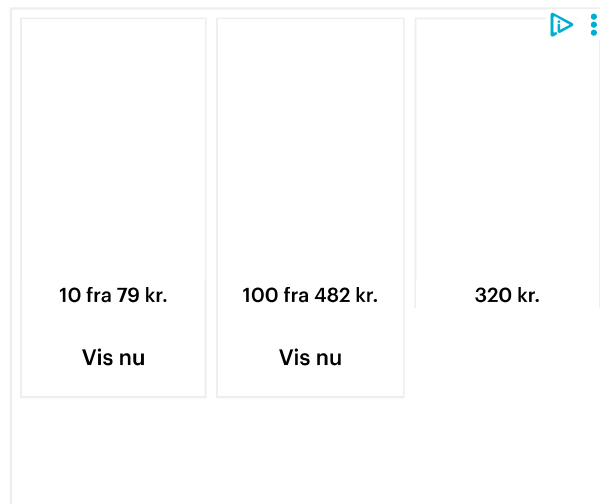
```
capitalCities.clear();
```

[Try it Yourself »](#)

ADVERTISEMENT



ADVERTISEMENT

[REMOVE ADS](#)

## HashMap Size

To find out how many items there are, use the `size()` method:

### Example

```
capitalCities.size();
```

[Try it Yourself »](#)



# Loop Through a HashMap

Loop through the items of a **HashMap** with a **for-each** loop.

**Note:** Use the **keySet()** method if you only want the keys, and use the **values()** method if you only want the values:

## Example

```
// Print keys
for (String i : capitalCities.keySet()) {
    System.out.println(i);
}
```

[Try it Yourself »](#)

## Example

```
// Print values
for (String i : capitalCities.values()) {
    System.out.println(i);
}
```

[Try it Yourself »](#)

## Example

[Try it Yourself »](#)

## Other Types

Keys and values in a **HashMap** are actually objects. In the examples above, we used objects of type "String". Remember that a **String** in Java is an object (not a primitive type). To use other types, such as **int**, you must specify an equivalent wrapper class: **Integer**. For other primitive types, use: **Boolean** for boolean, **Character** for char, **Double** for double, etc:

## Example

Create a **HashMap** object called **people** that will store **String** **keys** and **Integer** **values**:

```
// Import the HashMap class
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {

        // Create a HashMap object called people
        HashMap<String, Integer> people = new HashMap<String, Integer>();

        // Add keys and values (Name, Age)
        people.put("John", 32);
        people.put("Steve", 30);
        people.put("Angie", 33);

        for (String i : people.keySet()) {
            System.out.println("key: " + i + " value: " + people.get(i));
        }
    }
}
```



## When Order Matters

In the next chapter, you will learn about [TreeMap](#), which stores key/value pairs **in sorted order by key**.

## The var Keyword

From Java 10, you can use the **var** keyword to declare a **HashMap** variable without writing the type twice. The compiler figures out the type from the value you assign.

This makes code shorter, **but many developers still use the full type for clarity**. Since **var** is valid Java, you may see it in other code, so it's good to know that it exists:

### Example

```
// Without var
HashMap<String, String> capitalCities = new HashMap<String, String>();

// With var
var capitalCities = new HashMap<String, String>();
```

Try it Yourself »

## The Map Interface



```
import java.util.Map;
import java.util.HashMap;

Map<String, String> capitalCities = new HashMap<>();
```

Try it Yourself »

This means the variable (capitalCities) is declared as a **Map** (the interface), but it stores a **HashMap** object (the actual map). Since **HashMap** implements the **Map** interface, this is possible.

It works the same way, but some developers prefer this style because it gives them more flexibility to change the type later.

## Complete HashMap Reference

For a complete reference of HashMap methods, go to our [Java HashMap Reference](#).

### Exercise ?

#### A HashMap

- ☐ Stores multiple items of the same type
- ☐ Stores items in a queue
- ☐ Stores items in key/value pairs
- ☐ Stores items of the same type and can dynamically grow in size

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)[C](#)[< Previous](#)[Sign in to track progress](#)[Next >](#)

ADVERTISEMENT



[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)[C](#)

## COLOR PICKER



ADVERTISEMENT

YOUR CRM is outdated.  
Time to move on.

Start free trial →

Sales dashboard

Closest to your sales target

Progress view

\$4

Score by stage per rep

John Doe

Tony Brown

David Lee

Michael Smith

REMOVE ADS

ADVERTISEMENT

ADVERTISEMENT



Tutorials ▼

References ▼

Exercises ▼



Sign In

- ☰
- CSS
- JAVASCRIPT
- SQL
- PYTHON
- JAVA
- PHP
- HOW TO
- W3.CSS
- C



PLUS

SPACES

GET CERTIFIED

FOR TEACHERS

FOR BUSINESS

CONTACT US

Top Tutorials

- HTML Tutorial
- CSS Tutorial
- JavaScript Tutorial
- How To Tutorial
- SQL Tutorial
- Python Tutorial
- W3.CSS Tutorial
- Bootstrap Tutorial
- PHP Tutorial
- Java Tutorial
- C++ Tutorial
- jQuery Tutorial

Top References

- HTML Reference
- CSS Reference
- JavaScript Reference
- SQL Reference



AngularJS Reference

jQuery Reference

### Top Examples

- HTML Examples
- CSS Examples
- JavaScript Examples
- How To Examples
- SQL Examples
- Python Examples
- W3.CSS Examples
- Bootstrap Examples
- PHP Examples
- Java Examples
- XML Examples
- jQuery Examples

### Get Certified

- HTML Certificate
- CSS Certificate
- JavaScript Certificate
- Front End Certificate
- SQL Certificate
- Python Certificate
- PHP Certificate
- jQuery Certificate
- Java Certificate
- C++ Certificate
- C# Certificate
- XML Certificate



FORUM

ABOUT

ACADEMY

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.

Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness of all content. While using W3Schools, you agree to have read and accepted our [terms of use](#), [cookie and privacy policy](#).

[Copyright 1999-2025](#) by Refsnes Data. All Rights Reserved. [W3Schools is Powered by W3.CSS](#).